Process Management in Linux

What is Process?

- In Linux, a **process** is an instance (a step, stage, or situation) of a program that is currently being executed.
- It is a fundamental concept in operating systems and represents the running state of a program, including the program's code, current activity, and various resources allocated to it by the system.

Key Aspects of a Process in Linux:

Process ID (PID):

- Each process is assigned a unique identification number called the **PID**. This helps the operating system manage and track the process.

Parent and Child Processes:

- Every process in Linux, except the initial one (called **init** or **systemd**), is created by another process. The creating process is called the **parent**, and the newly created one is the **child**. The relationship between these processes is essential for process management.

States of a Process:

A process can be in different states during its life cycle:

- **Running:** Actively being executed by the CPU.
- **Waiting (or sleeping):** Waiting for an event or resource to continue.
- **Stopped:** Halted, often by receiving a signal.
- **Zombie:** The process has completed execution but has not been cleaned up by its parent.

Process Control Block (PCB):

- The operating system keeps information about each process in a data structure known as the **PCB**. This contains details like process state, PID, memory usage, open files, etc.

Context Switching:

- The OS can switch between processes to ensure multitasking. This process is called **context switching**, where the CPU saves the state of the current process and loads the state of another process.

Foreground and Background Processes:

- Processes can be run in the foreground (interactively) or in the background (running without user interaction). Background processes are often managed using the **&** operator, and the **jobs** command can be used to manage them.

Types of Processes:

- **Daemon:** A background process that runs without user interaction, often providing services like logging or network connections.
- **Interactive:** A process started by a user at the terminal, often running interactively in the foreground.

Common Commands for Process Management:

- `ps`: Lists the current processes running.
- `top` or `htop`: Provides a dynamic, real-time view of running processes.
- `kill`: Sends signals to processes, often used to terminate them.
- `nice`/`renice`: Adjusts the priority of processes.
- `bg`, `fg`, `jobs`: Manage background and foreground processes.

When a Process is Initialized:

A process is initialized in Linux when:

System Boot-Up: The initial `init` or `systemd` process is launched.

User Initiates a Command: Each time a user runs a command or program (e.g., launching an application, running a script, etc.), a new process is created by the terminal or the shell.

System Daemon Starts a Task: Daemons (background processes like a web server or cron job scheduler) can start other processes as needed.

Example

For example, when you type `ls` in the terminal, the shell (which is a process) uses `fork()` to create a child process. The child process then replaces its code with the code for `ls` using `exec()`, and the `ls` command is executed.

What is Bash Shell?

- Bash (Bourne Again SHell) is a Unix shell and command language, offering both an interactive command-line interface and a scripting environment.
- It serves as the default shell in many Linux distributions, including Linux Mint.

Features:

Command-Line Interface (CLI): Facilitates interaction with the operating system using textual commands rather than graphical interfaces.

Scripting Capabilities: Allows for the creation of shell scripts—files containing a series of commands that can be executed in sequence to automate tasks.

Job Control: Manages background and foreground processes, allowing users to suspend, resume, or terminate jobs.

Tab Completion & History: Enhances user experience by offering command and filename autocompletion, and maintaining a history of commands for easy retrieval.

Command Syntax:

Commands generally follow this structure:

```
command [options] [arguments]
```

Options modify the behaviour of the command (e.g., `-l` for a long listing in `ls -l`), while **arguments** specify the target (e.g., files or directories).

Environment Variables:

Environment variables (e.g., `PATH`, `HOME`) define the operating environment for the shell and can influence the behaviour of commands and scripts.

```
env
echo $HOME
echo $PATH
export HOME=/new/home/directory    //Setting Environment Variables
```

Configuration Files:

Configuration files like `~/.bashrc` and `~/.bash_profile` allow users to customize their shell environment by setting aliases, environment variables, and functions.

Scripting:

Bash scripting allows for powerful automation and process management. Key elements include:

Variables: Store values for later use (e.g., `myVar="Hello"`).

Control Structures: Implement conditional execution (`if` statements) and looping (`for`, `while` loops).

Functions: Reusable blocks of code to streamline scripts and improve maintainability.

Practical Applications:

System Administration: Perform maintenance tasks, manage services, and configure system settings efficiently.

Development Workflows: Compile code, manage dependencies, and automate testing and deployment processes.

Data Processing: Utilize text processing tools (`awk`, `sed`, `grep`) to manipulate and analyze data files.

Advanced Techniques:

Piping and Redirection: Use pipes (`|`) to pass the output of one command as input to another, and redirection (`>`, `>>`) to send output to files instead of the terminal.

Command Substitution: Execute a command within another command using backticks or `$()`, allowing for dynamic input.

Checking the Default Shell

To check the current default shell for a user, you can run:

```
echo $SHELL
```

If Bash is the default, it will return something like:

```
/bin/bash
```

Introduction to Processes and Basic Commands

Viewing Active Processes

Objective: Learn to view the list of active processes and understand their basic information.

Commands to Cover:

ps: Display currently running processes.

ps aux: Show all running processes with detailed information.

top: Display an interactive view of system processes in real-time.

The **`ps aux`** command in Linux is used to display information about all running processes on the system. Here's a detailed breakdown of each part of the command:

Command Breakdown

- **ps:** Process status command that displays information about running processes.
- **a:** Shows processes for **all users** (not just the current user).
- **u:** Displays the processes with more detailed information, including the username, CPU usage, and memory usage.
- **x:** Shows processes **not attached** to a terminal (background processes).

Output Columns

When you run **`ps aux`**, the following columns are displayed:

1. **USER**: The name of the user who owns the process.

2. **PID**: The Process ID, a unique identifier for each running process.
3. **%CPU**: The percentage of CPU time used by the process.
4. **%MEM**: The percentage of memory used by the process.
5. **VSZ**: Virtual memory size (in kilobytes). This includes all memory that the process can access, including memory that is swapped out or allocated but not used.
6. **RSS**: Resident Set Size. This is the non-swapped physical memory that a process is using (in kilobytes).
7. **TTY**: The terminal associated with the process. If the process is not running in a terminal, you'll see a **?**
8. **STAT**: The process state code:

    - **R:** Running
    - **S:** Sleeping (waiting for an event)
    - **D:** Uninterruptible sleep (usually I/O)
    - **T:** Stopped (either by a job control signal or because it is being traced)
    - **Z:** Zombie process (terminated but not cleaned up by its parent)
    - **s:** Session leader
    - **I:** Idle process
    - **<:** High-priority process
    - **l:** Multithreaded
    - Other codes may indicate flags like `+` (foreground process), `s` (session leader), and more.

9. **START**: The time the process started.
10. **TIME**: Total CPU time used by the process since it started.
11. **COMMAND**: The command or name of the process that started it, including any options or arguments.

Exercise:

1) Use ps to list the processes started by the current user. Identify the Process ID (PID) and the parent process (PPID) for at least two processes.

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ps
    PID TTY          TIME CMD
   3407 pts/0    00:00:00 bash
   3414 pts/0    00:00:00 ps
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
abdulla+    3407    3400  0 18:47 pts/0    00:00:00 bash
abdulla+    3416    3407 99 18:48 pts/0    00:00:00 ps -f
```

2) Run ps aux and analyze the output for system-wide processes. What do the following columns represent: USER, PID, %CPU, %MEM, TIME, COMMAND?

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  2.5  0.5  23300 14760 ?        Ss   18:45   0:04 /sbin/init splash
root           2  0.0  0.0      0     0 ?        S    18:45   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    18:45   0:00 [pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   18:45   0:00 [kworker/R-rcu_gp]
root           5  0.0  0.0      0     0 ?        I<   18:45   0:00 [kworker/R-sync_wq]
root           6  0.0  0.0      0     0 ?        I<   18:45   0:00 [kworker/R-kvfree_rcu_reclaim]
root           7  0.0  0.0      0     0 ?        I<   18:45   0:00 [kworker/R-slub_flushwq]
root           8  0.0  0.0      0     0 ?        I<   18:45   0:00 [kworker/R-netns]
root           9  0.0  0.0      0     0 ?        I    18:45   0:00 [kworker/0:0-cgroup_bpf_destroy]
root          10  0.2  0.0      0     0 ?        I    18:45   0:00 [kworker/0:1-events]
root          11  0.0  0.0      0     0 ?        I<   18:45   0:00 [kworker/0:0H-events_highpri]
root          12  0.0  0.0      0     0 ?        I    18:45   0:00 [kworker/u512:0-ipv6_addrconf]
root          13  0.0  0.0      0     0 ?        I<   18:45   0:00 [kworker/R-mm_percpu_wq]
root          14  0.0  0.0      0     0 ?        S    18:45   0:00 [ksoftirqd/0]
root          15  0.4  0.0      0     0 ?        I    18:45   0:00 [rcu_preempt]
root          16  0.0  0.0      0     0 ?        S    18:45   0:00 [rcu_exp_par_gp_kthread_worker/1]
root          17  0.0  0.0      0     0 ?        S    18:45   0:00 [rcu_exp_gp_kthread_worker]
root          18  0.0  0.0      0     0 ?        S    18:45   0:00 [migration/0]
root          19  0.0  0.0      0     0 ?        S    18:45   0:00 [idle_inject/0]
root          20  0.0  0.0      0     0 ?        S    18:45   0:00 [cpuhp/0]
root          21  0.0  0.0      0     0 ?        S    18:45   0:00 [cpuhp/1]
root          22  0.0  0.0      0     0 ?        S    18:45   0:00 [idle_inject/1]
root          23  0.4  0.0      0     0 ?        S    18:45   0:00 [migration/1]
root          24  0.0  0.0      0     0 ?        S    18:45   0:00 [ksoftirqd/1]
root          25  0.0  0.0      0     0 ?        I    18:45   0:00 [kworker/1:0-cgroup_free]
root          26  0.0  0.0      0     0 ?        I<   18:45   0:00 [kworker/1:0H-kblockd]
root          27  0.0  0.0      0     0 ?        S    18:45   0:00 [cpuhp/2]
root          28  0.0  0.0      0     0 ?        S    18:45   0:00 [idle_inject/2]
root          29  0.4  0.0      0     0 ?        S    18:45   0:00 [migration/2]
```

3) Launch top, monitor the CPU usage, and find the top 5 CPU-consuming processes. Press q to exit.

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ top

top - 18:48:58 up 3 min,  1 user,  load average: 0.46, 0.58, 0.26
Tasks: 387 total,   1 running, 386 sleeping,   0 stopped,   0 zombie
%Cpu(s):  3.8 us,  5.8 sy,  0.0 ni, 88.5 id,  0.0 wa,  0.0 hi,  1.9 si,  0.0 st
MiB Mem :   2852.7 total,    307.5 free,   1143.2 used,   1602.7 buff/cache
MiB Swap:   2852.0 total,   2852.0 free,      0.0 used.   1709.5 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   3421 abdulla+  20   0   14524   5736   3508 R  23.1   0.2   0:00.05 top
    105 root      20   0       0      0      0 I   7.7   0.0   0:00.43 kworker/2:1-events
      1 root      20   0   23300  14760   9876 S   0.0   0.5   0:04.50 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.07 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
      9 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0-cgroup_bpf_destroy
     10 root      20   0       0      0      0 I   0.0   0.0   0:00.37 kworker/0:1-events
     11 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     12 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/u512:0-ipv6_addrconf
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.05 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.75 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.04 rcu_exp_par_gp_kthread_worker/1
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.01 rcu_exp_gp_kthread_worker
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.01 migration/0
     19 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
```

Starting and Stopping Processes

Objective: Learn how to start, stop, and kill processes.

Commands to Cover:

&: Run a command in the background.

kill [PID]: Terminate a process by its PID.

kill -9 [PID]: Forcefully terminate a process.

pkill [name]: Terminate all processes by name.

Exercise:

1) Start the gedit text editor in the background using gedit &. Verify that it's running using ps.

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ gedit &
[1] 3720
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ps aux | grep gedit
abdulla+    3720 10.6  2.1 703992 61580 pts/0    Sl   18:54   0:02 gedit
abdulla+    3732  0.0  0.0   9152  2324 pts/0    S+   18:54   0:00 grep --color=auto gedit
```

2) Stop the gedit process using kill [PID] and verify that it has stopped using ps.

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ kill 3720
bash: kill: (3720) - No such process
[1]+  Done                    gedit
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ps aux | grep gedit
abdulla+    3747  0.0  0.0   9152  2328 pts/0    S+   18:56   0:00 grep --color=auto gedit
```

3) Use pkill gedit to stop any running instances of gedit (if they are open again). Verify using ps aux.

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ gedit &
[1] 3762
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ pkill gedit
[1]+  Done                    gedit
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ps aux | grep gedit
abdulla+    3776  0.0  0.0   9152  2324 pts/0    S+   18:57   0:00 grep --color=auto gedit
```

Managing Process Priority

In Linux, process priorities are managed using a mechanism called **niceness**. Each process has a priority, and the kernel uses this to determine how much CPU time each process gets. **Niceness** affects this priority, allowing users to influence how CPU-intensive processes are scheduled.

Process Priorities

Priority (PRI): A process's priority is a numerical value used by the kernel scheduler to determine how much CPU time to allocate. Lower numbers indicate higher priority.

Niceness (NICE): The niceness value (ranging from -20 to 19) is a way for users to influence the process's priority. A process with a lower niceness value (-20) has a higher priority, and a process with a higher niceness value (19) has a lower priority.

The default niceness value for processes is 0.

Niceness and Its Impact

Niceness Values:

- **-20:** Highest priority (most favorable scheduling).
- **0:** Default priority.
- **19:** Lowest priority (least favorable scheduling).

Effect on CPU Scheduling: A process with a lower niceness (negative) will get more CPU time compared to a process with a higher niceness (positive).

Viewing Niceness

You can view the niceness of running processes using the `ps` or `top` command. The `NI` column in the output of these commands shows the niceness value.

Example using **`ps`**:

```
ps -eo pid,ni,comm
```

Output:

```
  PID  NI COMMAND
    1   0 init
 1065   0 sshd
 1392   5 apache2
 1407  10 mysqld
```

Let's break down each part of the command:

Command Breakdown

ps: This stands for "process status." It is a command used to display information about the currently running processes on the system.

-e: This option tells `ps` to display information about all processes running on the system. It is equivalent to the `-A` option.

-o: This option allows you to specify the output format. You can choose which columns of information to display.

pid: This stands for **Process ID**. It is a unique identifier assigned by the operating system to each running process.

ni: This stands for **niceness**. The niceness value of a process indicates its priority. Lower values mean higher priority. The range is typically from `-20` (highest priority) to `19` (lowest priority). A niceness value of `0` is the default.

comm: This refers to the **command name** that started the process. It shows the name of the executable file or command that is running.

Changing Niceness (Renicing)

renice is used to change the niceness of an already running process.

Basic Syntax

```
renice [niceness value] -p [PID]
```

Example

```
renice 10 -p 1392
```

This command changes the niceness of the process with PID `1392` (e.g., `apache2`) to `10`, which lowers its priority.

Flags for `renice`

- **-p (PID):** Used to specify the process by its Process ID.
- **-u (User):** To change the niceness of all processes belonging to a specific user.
- **-g (Group):** To change the niceness of all processes in a group.

Example with user flag:

```
renice 5 -u aamer
```

This would renice all processes owned by the user `aamer` to a niceness value of `5`.

Who Can Change Niceness?

Normal users can only increase the niceness value (i.e., make processes run with lower priority).

Root (superuser) can both increase and decrease the niceness of any process.

Real-time and Niceness

Processes that are running with real-time scheduling priorities are not affected by niceness. Niceness only affects the scheduling of normal (non-real-time) processes.

Exercise

1) Run the `top` command and identify the default niceness (NI) value for processes.

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ top

top - 13:08:43 up 1 min,  1 user,  load average: 1.17, 0.69, 0.27
Tasks: 364 total,   1 running, 363 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.3 us,  0.3 sy,  0.0 ni, 99.2 id,  0.0 wa,  0.0 hi,  0.1 si,  0.0 st
MiB Mem :   2852.6 total,    208.8 free,   1214.4 used,   1629.7 buff/cache
MiB Swap:   2852.0 total,   2852.0 free,      0.0 used.   1638.2 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2477 abdulla+  20   0 4207800 286924 133756 S   2.0   9.8   0:07.11 gnome-shell
   2813 abdulla+  20   0  214292  37860  30024 S   0.7   1.3   0:00.38 vmtoolsd
   3379 abdulla+  20   0  701580  60128  48452 S   0.7   2.1   0:00.94 gnome-terminal-
   3429 abdulla+  20   0   14524   6020   3792 R   0.7   0.2   0:00.31 top
    548 root      20   0       0      0      0 I   0.3   0.0   0:00.06 kworker/0:3-events
    790 root      20   0  244760   9496   8076 S   0.3   0.3   0:00.33 vmtoolsd
      1 root      20   0   23308  14528   9768 S   0.0   0.5   0:02.08 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.03 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
      9 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0-events
     10 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/0:1-events
     11 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     12 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/u512:0-ipv6_addrconf
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.07 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.27 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.01 rcu_exp_par_gp_kthread_worker/1
```

2) Start a CPU-intensive process using the command `yes > /dev/null &`. Use `top` to monitor its behavior. What do you observe about CPU usage and priority?

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ yes > /dev/null &
[1] 3491
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ top

top - 13:10:42 up 3 min,  1 user,  load average: 0.69, 0.60, 0.29
Tasks: 365 total,   2 running, 363 sleeping,   0 stopped,   0 zombie
%Cpu(s):  4.3 us, 12.2 sy,  0.0 ni, 83.6 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   2852.6 total,    230.4 free,   1192.6 used,   1630.0 buff/cache
MiB Swap:   2852.0 total,   2852.0 free,      0.0 used.   1660.0 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   3491 abdulla+  20   0    8296   2052   1932 R  99.7   0.1   0:46.18 yes
   2477 abdulla+  20   0 4207908 287292 133884 S   0.3   9.8   0:08.33 gnome-shell
   3495 abdulla+  20   0   14524   5984   3752 R   0.3   0.2   0:00.06 top
      1 root      20   0   23308  14528   9768 S   0.0   0.5   0:02.09 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.03 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
      9 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0-events
     10 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/0:1-events
     11 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     12 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/u512:0-ipv6_addrconf
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.07 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.29 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.01 rcu_exp_par_gp_kthread_worker/1
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.01 rcu_exp_gp_kthread_worker
```

3) Start the same command `yes > /dev/null &` with a niceness value of `10` using `nice -n 10 yes > /dev/null &`. Monitor the difference in behavior using `top`.

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nice -n 10 yes > /dev/null &
[2] 3509
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ top

top - 13:15:33 up 8 min,  1 user,  load average: 1.92, 1.26, 0.64
Tasks: 319 total,   3 running, 316 sleeping,   0 stopped,   0 zombie
%Cpu(s):  4.7 us, 27.2 sy,  5.2 ni, 62.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   2852.6 total,    231.7 free,   1189.9 used,   1631.5 buff/cache
MiB Swap:   2852.0 total,   2852.0 free,      0.0 used.   1662.7 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   3491 abdulla+  20   0    8296   2052   1932 R 100.0   0.1   5:37.38 yes
   3509 abdulla+  30  10    8296   2048   1932 R 100.0   0.1   2:32.15 yes
   3511 abdulla+  20   0   14524   5976   3748 R   0.7   0.2   0:00.21 top
    790 root      20   0  244760   9532   8076 S   0.3   0.3   0:00.81 vmtoolsd
   2477 abdulla+  20   0 4207920 287256 133884 S   0.3   9.8   0:10.44 gnome-shell
      1 root      20   0   23308  14528   9768 S   0.0   0.5   0:02.10 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.03 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     11 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     12 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/u512:0-ipv6_addrconf
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.08 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.31 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.01 rcu_exp_par_gp_kthread_worker/1
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.01 rcu_exp_gp_kthread_worker
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.00 migration/0
     19 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
```

4) Change the priority of the first `yes` process using `renice` to `15`. Verify the change using `top` and observe the effect on CPU usage.

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ sudo renice 15 -p 3509
[sudo] password for abdullah-sohail:
3509 (process ID) old priority 10, new priority 15
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ top

top - 13:18:33 up 11 min,  1 user,  load average: 2.03, 1.62, 0.89
Tasks: 313 total,   3 running, 310 sleeping,   0 stopped,   0 zombie
%Cpu(s):  5.9 us, 30.0 sy,  4.7 ni, 59.4 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   2852.6 total,    227.8 free,   1192.8 used,   1632.6 buff/cache
MiB Swap:   2852.0 total,   2852.0 free,      0.0 used.   1659.9 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   3491 abdulla+  20   0    8296   2052   1932 R 100.0   0.1   8:37.50 yes
   3509 abdulla+  35  15    8296   2048   1932 R  99.7   0.1   5:32.03 yes
   2477 abdulla+  20   0 4207972 287344 133884 S   3.3   9.8   0:11.54 gnome-shell
   3379 abdulla+  20   0  701580  60348  48452 S   3.3   2.1   0:03.71 gnome-terminal-
     40 root      20   0       0      0      0 I   0.3   0.0   0:00.76 kworker/u514:0-events_unbound
      1 root      20   0   23308  14528   9768 S   0.0   0.5   0:02.10 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.03 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     11 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     12 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/u512:0-ipv6_addrconf
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.08 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.32 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.01 rcu_exp_par_gp_kthread_worker/1
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.01 rcu_exp_gp_kthread_worker
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.00 migration/0
     19 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ killall yes
[1]-  Terminated              yes > /dev/null
[2]+  Terminated              nice -n 10 yes > /dev/null
```